Java Institute for Advanced Technology

Department of Examinations

COURSE(S) – (LEADING TO)	BSC (HONS) IN SOFTWARE ENGINEERING
UNIT NAME	HANDHELD DEVICE PROGRAMMING II
UNIT ID	JIAT/HHDPII
EXAMINATION ID	JIAT/HHDPII/EX01

Java Institute for Advanced Technology

Department of Examinations

COURSE(S) – (LEADING TO)	BSC (HONS) IN SOFTWARE ENGINEERING
UNIT NAME	HANDHELD DEVICE PROGRAMMING II
UNIT ID	JIAT/HHDPII
EXAMINATION ID	JIAT/HHDPII/EX01

NAME	:	MORAWAKAGE AKINDA METHMIKA PERERA
STUDENT ID	:	2025/2026/BCU/SE/I1/054
NIC NO	:	200421803135

Acknowledgment

I would like to convey my personal gratitude to all lecturers and academic staff of the Java Institute of Advanced Technology that made knowledge, guidance, and academic support to complete this project possible. Their commitment towards the teaching and constant motivation to the degree program course have been of much importance in developing my technical and analytical skills.

I would like to particularly acknowledge the instructors of the Handheld Device Programming II unit, their valuable advice, constructive feedback, and guidance throughout the planning and development processes of the EcoTrekk system. I also wish to thank my colleagues and friends who contributed ideas, gave useful conversations, and supported me throughout the development and testing of this project.

Lastly, I should like to say my greatest appreciation goes out to my family who were patient and inspirational and never gave up on me in my academic life. The support and faith they showed in me made me have confidence and have the will to achieve this project.

Abstract

EcoTrekk is an all-inclusive, cloud-based camping gear rental solution. The system consists of three closely integrated units: a native Android mobile application developed in Java, a RESTful Spring Boot backend server developed in Java 17, and a React-based administrative web dashboard developed in TypeScript. Together, these units are intended to fill a gap in the outdoor recreation rental market that has been demonstrably underserved -the lack of a reliable, real-time, and offline capable digital system to manage outdoor camping gear across geographically dispersed rental hubs.

The Android application is used to allow customers to navigate through available gear inventories. Customers can authenticate with Android Biometric APIs and can complete a rental checkout flow which is secured by the Stripe payment processing. An exclusive Shake-to-Refresh function. achieved by using the device accelerometer, allows instant data synchronization at will. The Spring Boot back-end uses JWT-based stateless authentication, a transactional inventory auto-allocation engine which works to prevent simultaneous double booking, and a WebSocket endpoint which follows the STOMP sub-protocol. The React administrative dashboard, which uses Vite, builds the interface with Material Ul and analyses with Recharts, uses the REST API to gather information and the WebSocket channel to keep the state of inventory and orders up-to-date.

Critical analysis of the system in relation to its initial specifications shows that all primary functional requirements were achieved. The inventory lifecycle, biometric authentication and gesture-based synchronization were checked by automated testing and proved to be correct. Limitations found are lack of end-to-end load testing and the lack of a native iOS client. Future work plans include AI assisted predictive inventory management, a multi-tenant Hub architecture and cross platform mobile migration.

Table of Contents

1. Introduction 8
   1.1 Problem Statement 8
   1.2 Proposed Solution 8
2. Background 10
   2.1 Camping Gear Rental Market 10
   2.2 Technological Foundations 10
3. Specifications 11
   3.1 Stakeholder Analysis 11
   3.2 Functional Requirements 11
   3.3 Non-Functional Requirements 12
4. Design 13
   4.1 System Architecture Overview 13
   4.2 UML Components Diagram 13
   4.3 Module Decomposition 14
   4.4 Database Design – Entity Relationship Model 14
   4.5 UI Designs 17
5. Implementation: 24
   5.1 Android Application Implementation 24
   5.2 Spring Boot Backend Implementation 25
   5.3 React Admin Dashboard Implementation 26
6. Results and Evaluation: 27
   6.1 Inventory Lifecycle Testing 27
   6.2 Shake-to-Refresh Testing 27
   6.3 Biometric Authentication Testing 27
   6.4 WebSocket Real-Time Update Testing 27
   6.5 Future Testing Requirements 28
7. Future Work: 29
   7.1 AI-Assisted Predictive Inventory Management 29
   7.2 Multi-Tenant Hub Architecture 29
   7.3 Cross-Platform Mobile Client 29
8. Conclusions: 30
9. Reflection on Learning: 31
10. References: 32

List of Abbreviations

Abbreviation	Expansion
API	Application Programming Interface
ER	Entity Relationship
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
MUI	Material UI
ORM	Object Relational Mapping
REST	Representational State Transfer
UAT	User Acceptance Testing
UI	User Interface
UML	Unified Modelling Language
UX	User Experience

List of Tables

Table 1: Inventory Lifecycle Testing Outcomes 27

List of Figures

Figure 1: UML Components Diagram....13
Figure 2: ER Diagram....16
Figure 3: App Splash Screen....17
Figure 4: App Welcome Screen....17
Figure 5: App Signup Screen....17
Figure 6: App Wishlist Screen....17
Figure 7: App Login Screen....17
Figure 8: App Home Screen....17
Figure 9: App Gears Screen....18
Figure 10: App Single Hub Screen....18
Figure 11: App Hubs Screen....18
Figure 12: App Checkout Screen....18
Figure 13: App Cart Screen....18
Figure 14: App Single Gear Screen....18
Figure 15: App Checkout Success Screen....19
Figure 16: App Personal Information Screen....19
Figure 17: App Profile Screen....19
Figure 18: App Settings Screen....19
Figure 19: App Order Details Screen....19
Figure 20: App Order History Screen....19
Figure 21: Admin Dashboard Page....20
Figure 22: Admin Signin Page....20
Figure 23: Admin Admin Management Page....20
Figure 24: Admin Order Management Page....21
Figure 25: Admin Rental Hubs Management Page....21
Figure 26: Admin Users Management Page....21
Figure 27: Admin Gear Management Page....22
Figure 28: Admin Reports Page....22
Figure 29: Admin Profile Page....23
Figure 30: My Hub Page....23

1. Introduction

EcoTrekk comprises a three-part structure to operate its rental service. Outdoor enthusiasts and adventure travelers use EcoTrekk to access quality camping gear which is both reliable and environmentally conscious through this digital rental platform. EcoTrekk operates utilizing three separate elements:

- **Android Mobile Application** for end-users to browse, reserve, and manage gear rentals.
- **Spring Boot Backend** for business logic, data management, and integration.
- **React/TypeScript Admin Frontend** for administrative and operational tasks.

Through streamlined rental operations and better supply monitoring the platform enhances customers' overall experience. EcoTrekk exists because of our desire to enable sustainable outdoor adventure through efficient equipment sharing.

1.1 Problem Statement

The modern camping and outdoor recreation industry is marked by a large disparity between the demand and supply of the equipment. Potential customers will often have neither the funds nor the storage space to purchase quality equipment, but the current rental business is still largely analogue using telephone bookings, paper-based inventory control, and cash payment. The consequence of this paradigm is well-documented inefficiencies like booking two times per customer, incorrect inventory management and lack of proper communication with customers.

1.2 Proposed Solution

EcoTrekk responds to the stated inefficiencies of the traditional camping gear rental systems with multiple technological integrations, which are designed thoroughly. First, Android application is based on a mobile architecture which is developed using Java and newer Android development libraries to offer an efficient and responsive user interface to browse equipments, manage rentals and communicate with rental hubs. The app will interact with a centralized backend service to access equipment availability, complete reservations, and user transactions, so that customers can get access to correct and current information on rental.

Second, the system incorporates Google Maps Software Development Kit (SDK) to enable the system to have geospatial knowledge of the locations of the rental hubs. This option enables users to see the nearby equipment hubs and get navigation directions instantly using the application interface. A mapping functionality within the system enhances the accessibility and allows the EcoTrekk users to design equipment pickup routes in a more efficient manner.

Third, the site includes automated data synchronization systems that make the mobile application consistent with the backend system. The application uses controlled API

communication and background data refresh operations to retrieve the relevant latest inventory data and order updates stored by the backend services, and thus keep an adequate view of the system state to the customers and administrators.

On top of the basic functionality features, EcoTrekk is also able to showcase the capabilities of the Android platform to combine both hardware and software in building richer mobile experiences. The app uses various native device features such as the accelerator sensor which lets it use a shake-to-refresh gesture to easily refresh a gear listing, the telephony subsystem which lets a user contact any hub manager by dialing directly, the camera and media subsystem which lets a user pick and crop an image using the UCrop library, and the biometric authentication system which lets a user verify their identity securely by using the BiometricPrompt API. These integrations, combined, will not only meet the functional needs of the system, but also indicate how mobile platforms could provide a more interactive and context-driven user experience than a traditional web-based one based on the functions of the rental.

2. Background

2.1 Camping Gear Rental Market

The worldwide market for outdoor recreational equipment enjoyed continuing growth for more than ten years because consumers showed increasing interest toward immersive tourism and eco-friendly travel options. Information from the Outdoor Industry Association (2023) showed that outdoor activities including hiking and camping hit record participation levels after the pandemic but equipment ownership has lagged behind. The gap between increased participation and stagnant ownership rates created an emerging business opportunity for gear rental services that operate from peer-to-peer to business-to-consumer markets. The rental market faces substantial demand yet its operational technology systems appear extremely immature and ineffective. Most small-to-medium scale rental operations still rely on spreadsheets for managing inventory while conducting customer service through phone calls and payment processing via cash or basic card readers these methods have limited scalability and produce a high volume of operational errors.

2.2 Technological Foundations

The technology stack chosen to be used in EcoTrekk was selected to be representative of current industry practice and scholarly best practice in each of the domains within it. Android platform, based on Java and AndroidX library suite, offers robust access to hardware of the device such as biometric sensors. Java 17 Spring boot provides a framework that can be used to develop a RESTful API, that is well documented, production-proven, and Spring Security offers a full JWT authentication stack. Firebase Authentication is used as the identity provider of the customer-facing flow, which offers industry-standard open-source OAuth token handling that is later authenticated with the backend. React with TypeScript and Vite is a modern frontend development experience that is highly type-safe and the Recharts library delivers declarative data visualization elements that are appropriate in the administrative dashboard. Stripe is chosen as the payment gateway due to its rich SDK support of Android and highly documented webhook system to handle payment event in the server side.

3. Specifications

The purpose of the Specification and Design sections is to give the reader a clear picture of the system you plan to create, in terms of the capability required. A specification should tell the reader what the software system is required to do. The design then gives the top-level details of how the software system meets the requirement. It will also identify constraints on the software solution. Describing what a software system does (specification) and how it does so (design) effectively usually means describing it from more than one viewpoint. Each viewpoint will convey some information about the system that other viewpoints omit. (You would use the same technique when describing any complicated construction such as a building, an aircraft, a novel, or a painting)

3.1 Stakeholder Analysis

Three primary stakeholder categories have been identified through a requirement gathering process:

- **Customer (Camper):** An end-user who discovers rental hubs, browses and books gear, makes payments, tracks orders, and communicates with hub managers.
- **Hub Manager:** An operator who manages inventory, processes rental requests, and responds to customer enquiries via the React admin dashboard.
- **System Administrator:** A privileged operator who oversees the entire platform, manages hub registrations, monitors system health, and configures notification templates.

3.2 Functional Requirements

3.2.1 Customer-Facing Features:

- Account registration, login, and profile management with biometric authentication.
- Browse and search available rental hubs by geographic proximity using an embedded Google Maps interface.
- View detailed gear catalogues per hub, including availability, pricing, and high-resolution imagery.
- Submit rental orders with date-range selection, quantity specification, and Stripe-mediated payment processing.
- Track active and historical orders with real-time status updates delivered via push notification.
- Initiate direct telephone calls to hub managers via native telephony integration.
- Refresh gear listings through an accelerometer-detected shake gesture.

3.2.2 Administrative Features:

- Hub management: Create, update, and deactivate rental hub records, including geographic coordinates.
- Inventory management: Add, modify, and retire gear items with Cloudinary hosted images.
- Order management: View, approve, reject, and fulfil rental orders.
- Notification broadcasting: Compose and dispatch targeted push notifications to customer segments.

3.3 Non-Functional Requirements

3.3.1 Performance:

- The Android application shall load inventory data from the backend API on a standard 4G mobile connection on devices running Android 9 or above.

3.3.2 Reliability and Availability:

- The backend API shall maintain maximum uptime under normal single-instance deployment conditions.

3.3.3 Security

- All API communication shall be conducted over HTTPS.
- Authentication tokens (JWT) shall be stored exclusively in SharedPreferences with private mode, and shall be invalidated upon logout.
- Payment card data shall not be processed or stored within the application. All payment operations are delegated to the Stripe SDK.

3.3.4 Usability

- The application follows the Android Material Design 3 guidelines throughout all user-facing screens.
- The application shall be operatable with a single hand on a standard smartphone form factor.

3.3.5 Maintainability

- The codebase shall be structured according to well-established architectural patterns (Clean Architecture on Android, layered architecture in Spring Boot, component-based architecture in React) to facilitate future modification.

4. Design

4.1 System Architecture Overview

The EcoTrekk system follows a three-level client-server model, but with some additional specialized third-party service integrations. The three primary tiers are:

- **Android mobile app and the React web admin panel**, which are the presentation and application logic level.
- **Spring boot RESTful API**, which is the business logic and data coordination level.
- **MySQL relational database**, which is the persistent data level.

Firebase Cloud Messaging (notification delivery), Cloudinary (media asset management), Stripe (payment processing) and the Google Maps Platform (geospatial services) are added to these tiers.

4.2 UML Components Diagram

The component diagram below (Figure 4.2) illustrates the principal software components and their communication channels. The Android Application communicates with the Spring Boot Backend exclusively via HTTPS REST calls and, for authentication token refresh, via the Firebase Authentication SDK. The React Admin Dashboard communicates with the backend via both HTTPS REST and a persistent WebSocket connection using the STOMP sub-protocol. The backend in turn communicates with the MySQL database, the Stripe Payment API, and the Firebase Admin SDK for token verification

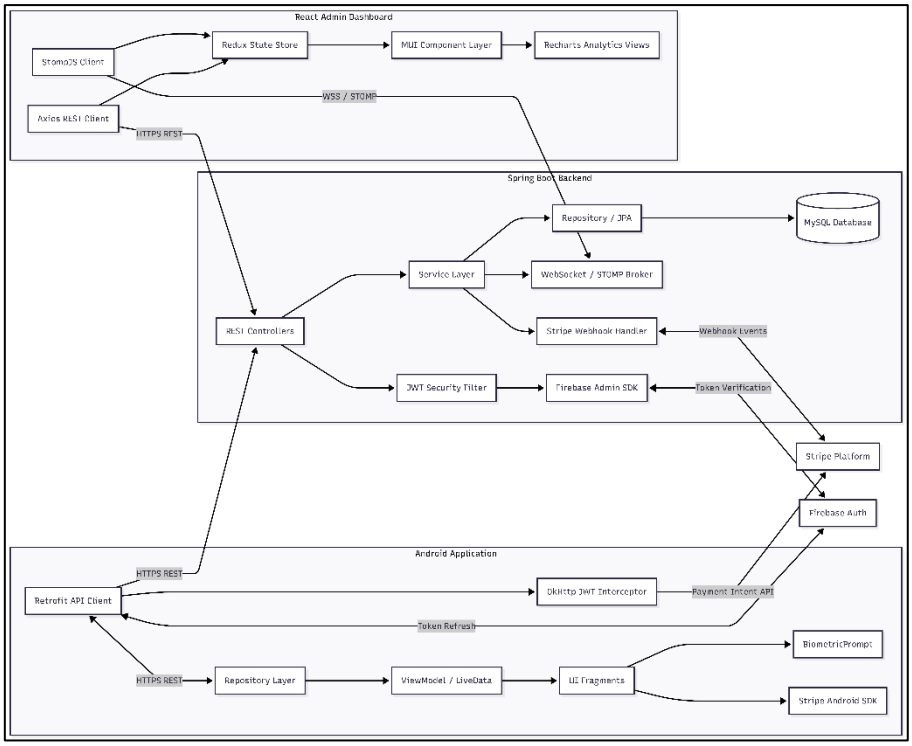


Figure 1: UML Components Diagram

4.3 Module Decomposition

4.5.1 Android Application Modules

- Auth - Firebase sign-in, token management, Biometric Prompt integration, OkHttp JWT interceptor.
- Hubs - Hub listing screen, map view, proximity sorting.
- Inventory - Inventory browsing, item detail screen, live availability display, search and category filtering.
- Checkout - Cart management, biometric confirmation, Stripe payment, confirmation screen.
- Orders - Order history screen, invoice detail screen, real-time status updates.

4.5.2 Spring Boot Backend Modules

- Security - JWT filter chain, Firebase Admin SDK integration, role-based access control.
- Hub - Hub entity, repository, service, REST controller.
- Inventory - Inventory entity, allocation service (transactional), REST controller.
- Order - Order entities, order creation service with atomic allocation, REST controller.
- Invoice - Invoice entity, Stripe webhook handler, payment status reconciliation.
- WebSocket - STOMP configuration, event publisher, subscription management.

4.5.3 React Dashboard Modules

- Auth — Admin login, JWT storage, Axios interceptor.
- Hubs — Hub management CRUD views.
- Inventory — Inventory table, real-time stock updates via STOMP.
- Orders — Order list, detail view, return processing form.
- Analytics — Recharts dashboard, revenue and stock utilisation charts.

4.4 Database Design – Entity Relationship Model

4.4.1 Entity Descriptions

1. User Management & Authentication

- **Customer:** Represents the end-users using the Android mobile app. It stores their personal details (name, email, phone) and links to their Firebase representation for authentication.
- **CustomerDeviceToken:** Stores Firebase Cloud Messaging (FCM) device tokens linked to specific Customers. This is essential for sending targeted push notifications to users' phones.
- **AdminUser:** Represents the top-level system administrators who have full access to the web dashboard to oversee the entire platform, manage hubs, and view global analytics.
- **HubManager:** Represents the staff members assigned to manage specific rental locations (Hubs). They process local inventory, handle physical gear returns, and manage local orders.

2. Catalog & Inventory (The "Gears")

- **GearCategory:** A classification table that groups camping equipment into logical sections (e.g., "Tents", "Sleeping Bags", "Cooking Gear") to make browsing easier on the mobile app.
- **Gear:** The core product table. It holds all static details about a specific piece of equipment (name, description, daily rental price, brand) and links to its respective GearCategory.
- **GearImage:** Stores URLs (likely pointing to Cloudinary) for the photos associated with a specific Gear.
- **Inventory:** The global tracker for physical items. Every single physical item in existence has a unique tracking record here, linked back to a generic Gear record.
- **Hub:** Represents a physical branch, store, or pickup location where camping gear is stored and where customers go to collect or return their rentals.
- **HubInventory:** A crucial mapping table linking actual Inventory items to specific Hubs. It tracks exactly which hub currently holds which physical gear and monitors its real-time availability status (Available, Rented, Damaged).

3. Shopping Flow

- **Cart:** Represents the active shopping session for a customer. A user can only have one active cart at a time.
- **CartItem:** The specific gears a user has added to their Cart. It outlines which gear they want, from which hub, and the dates they intend to rent it.
- **Wishlist:** A list of saved or favorite gears that a customer wants to keep track of for future trips.
- **WishlistItem:** The individual gear entries saved within a specific user's Wishlist.

4. Orders & Billing

- **Invoice:** The master record of a completed checkout. It acts as the "Order" and contains total pricing, Stripe payment references, the rental start/end dates, the destination Hub, and the current status of the order (Pending, Active, Completed, Cancelled).
- **InvoiceItem:** The individual line items within an Invoice. It locks in the specific piece of gear rented, the price at the time of purchase, and tracks the return status of that specific item.

5. System Utilities

- **Notification:** Stores the history of alerts sent to users or managers (e.g., "Order Confirmed", "Overdue Return"). It powers the in-app notification bell.

4.4.2 ER Diagram

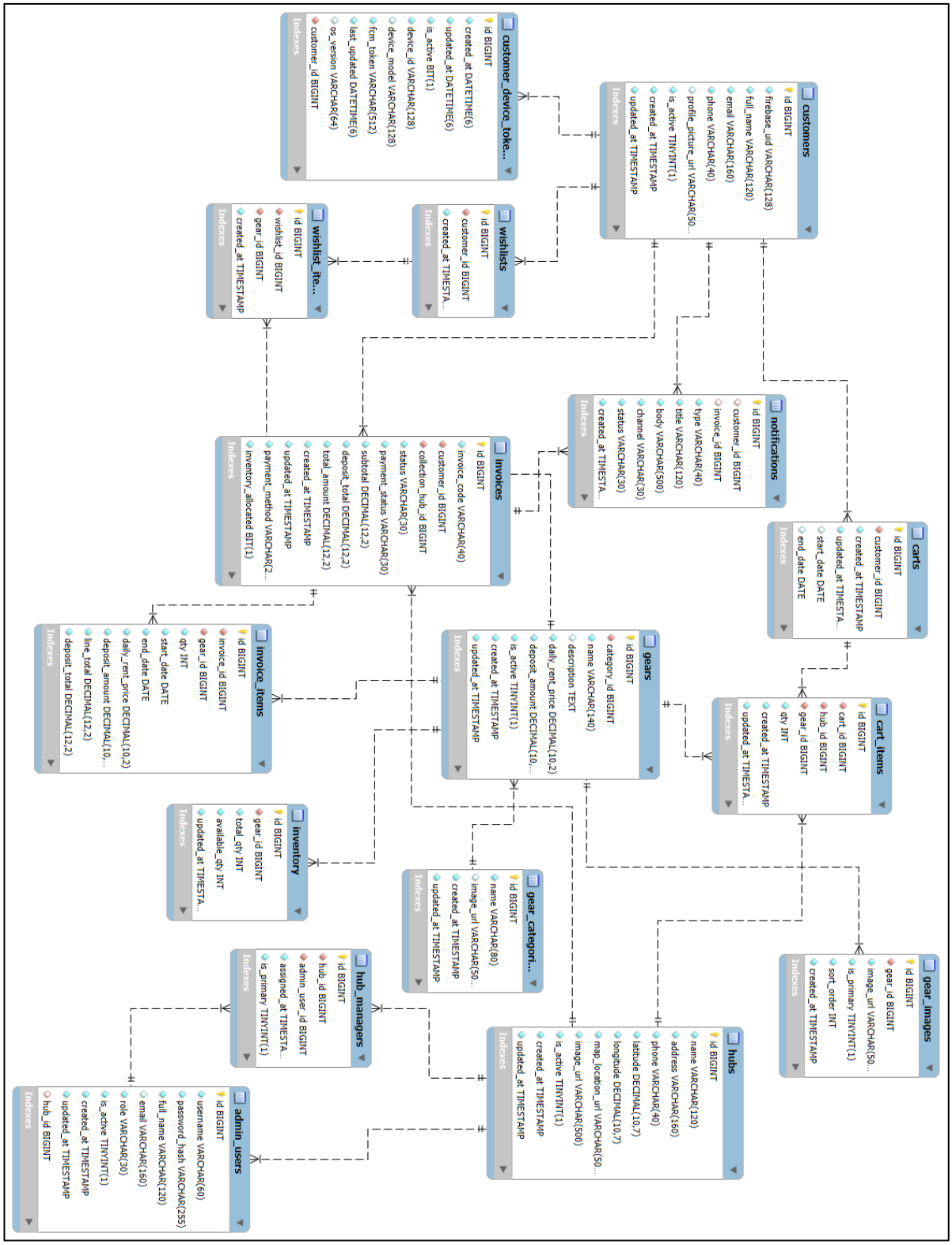


Figure 2: ER Diagram

4.5 UI Designs

4.5.1 EcoTrekk Mobile App



Figure 3: App Splash Screen



Figure 4: App Welcome Screen

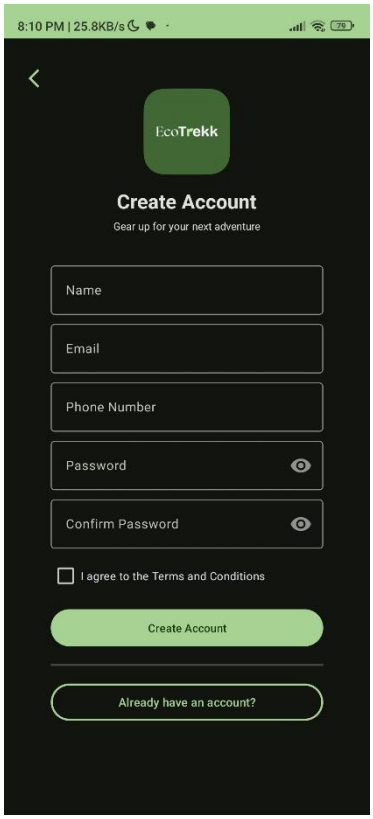


Figure 5: App Signup Screen

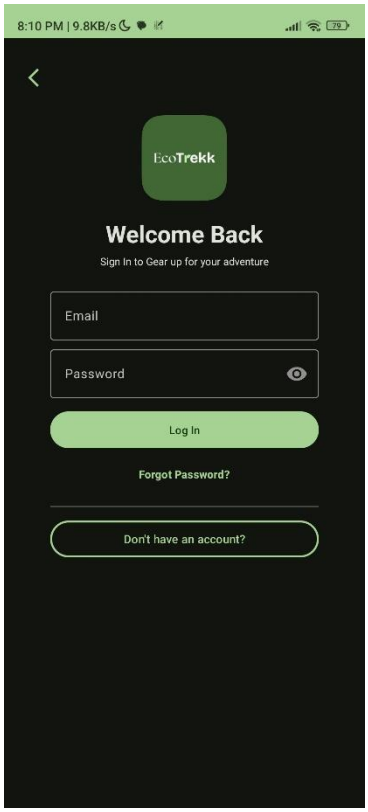


Figure 7: App Login Screen

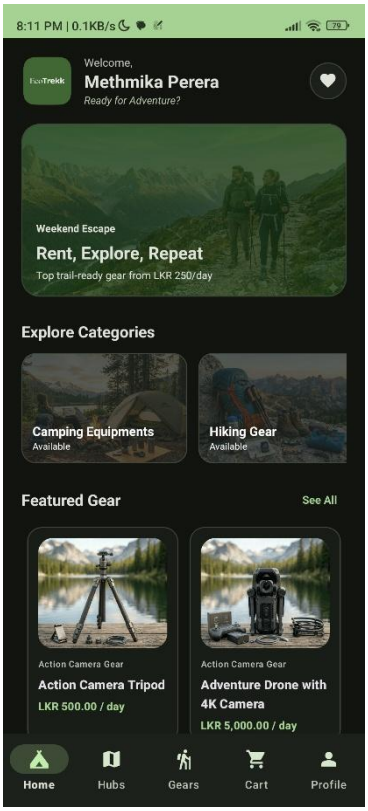


Figure 8: App Home Screen

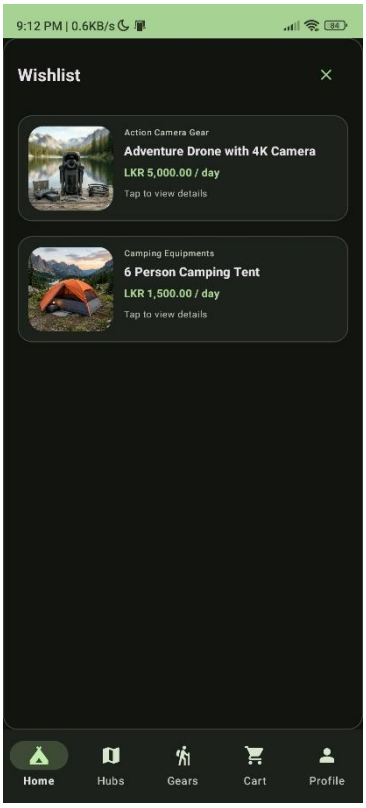


Figure 6: App Wishlist Screen

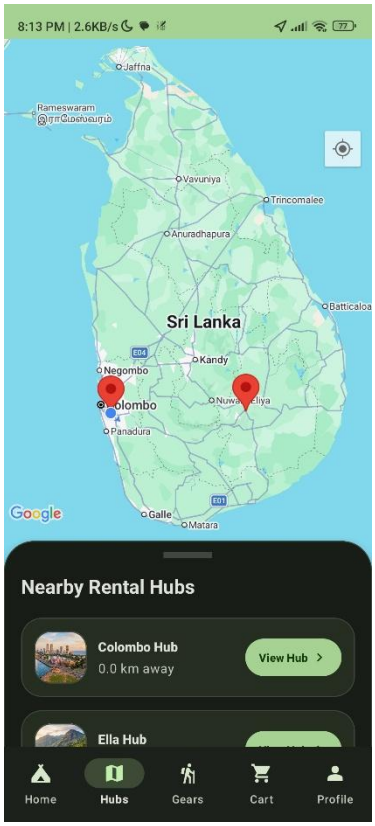


Figure 11: App Hubs Screen

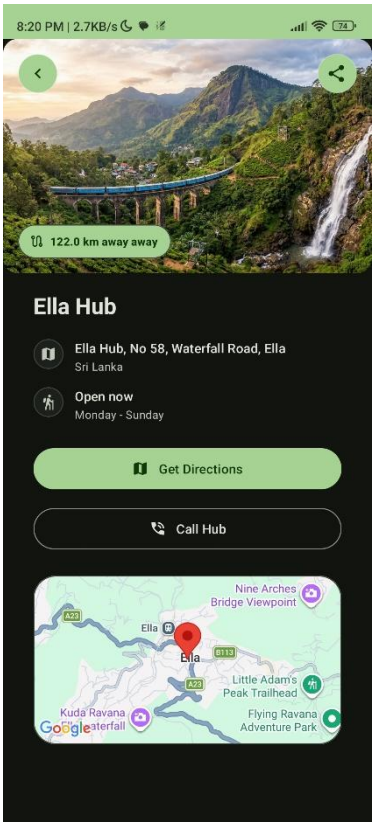


Figure 10: App Single Hub Screen

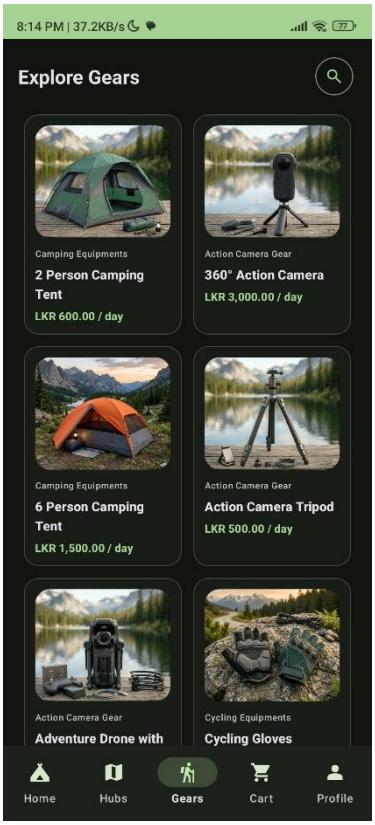


Figure 9: App Gears Screen



Figure 14: App Single Gear Screen

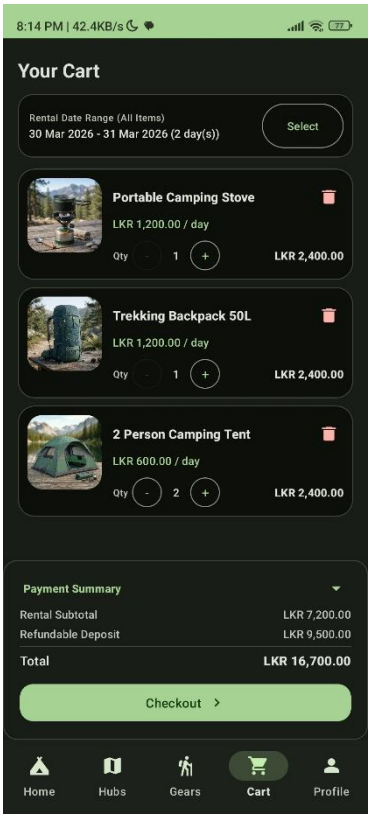


Figure 13: App Cart Screen

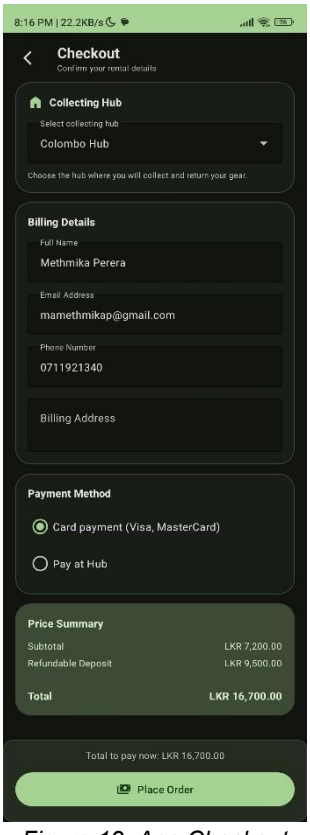


Figure 12: App Checkout Screen

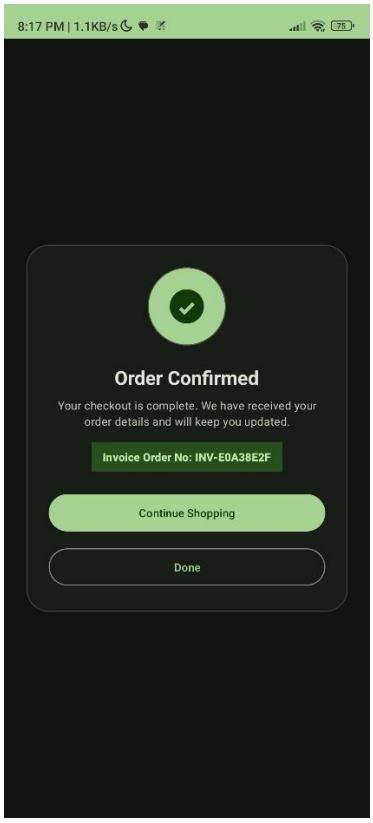


Figure 15: App Checkout Success Screen

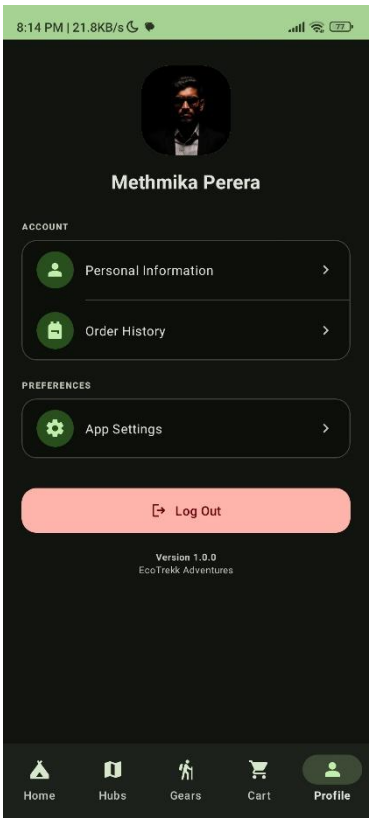


Figure 17: App Profile Screen

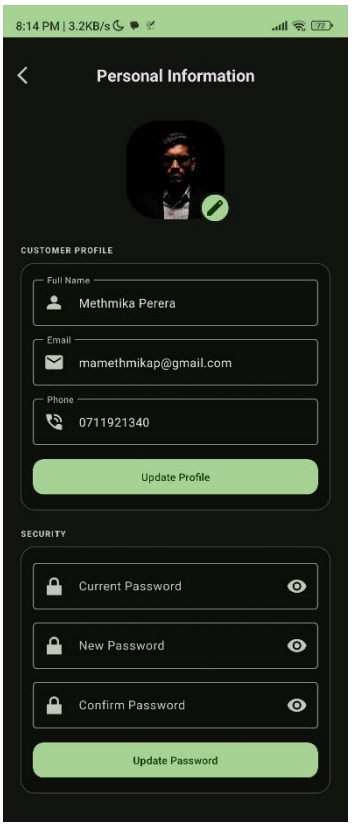


Figure 16: App Personal Information Screen

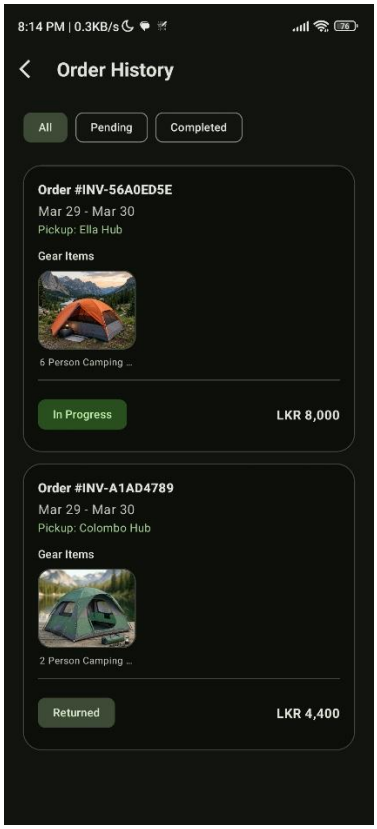


Figure 20: App Order History Screen

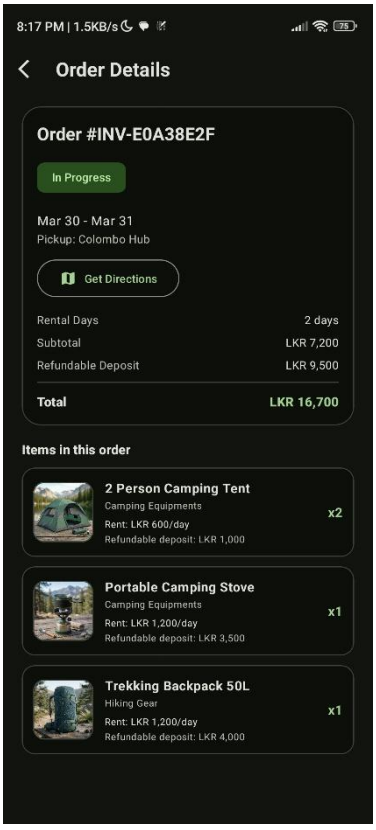


Figure 19: App Order Details Screen

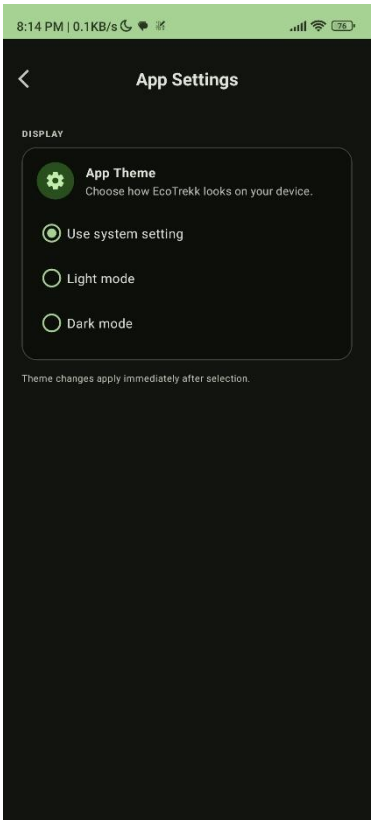


Figure 18: App Settings Screen

4.5.1 EcoTrekk Admin Web

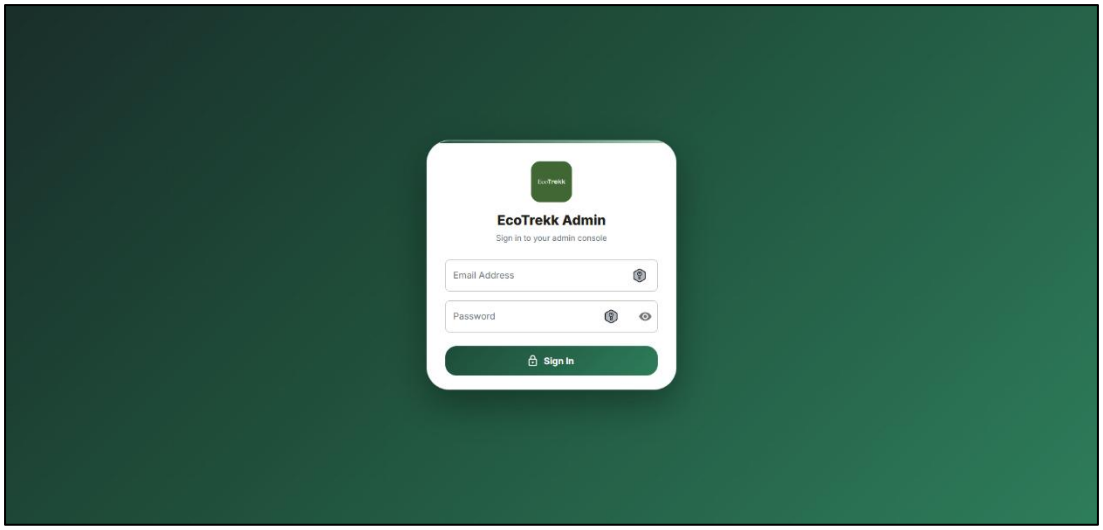


Figure 22: Admin Signin Page

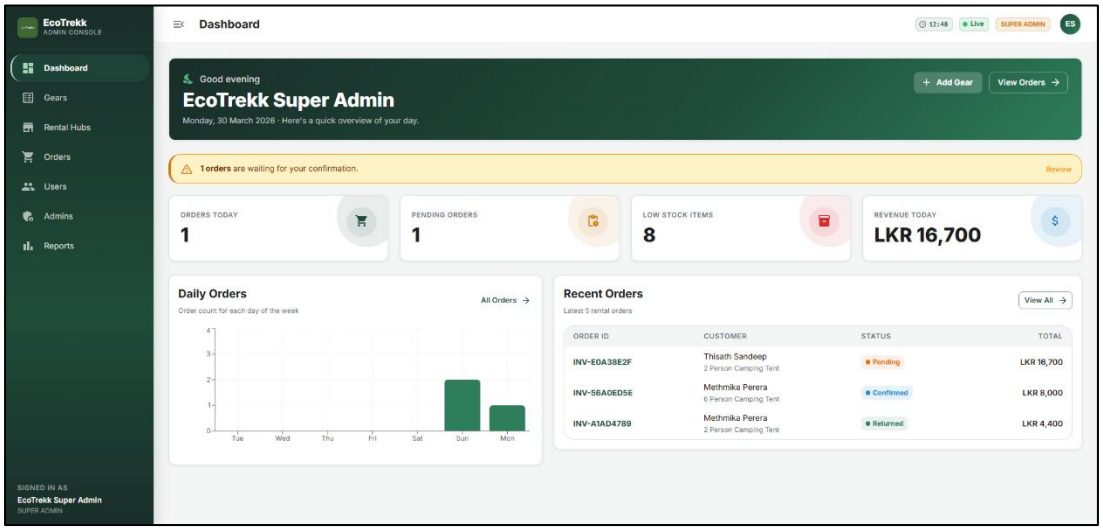


Figure 21: Admin Dashboard Page

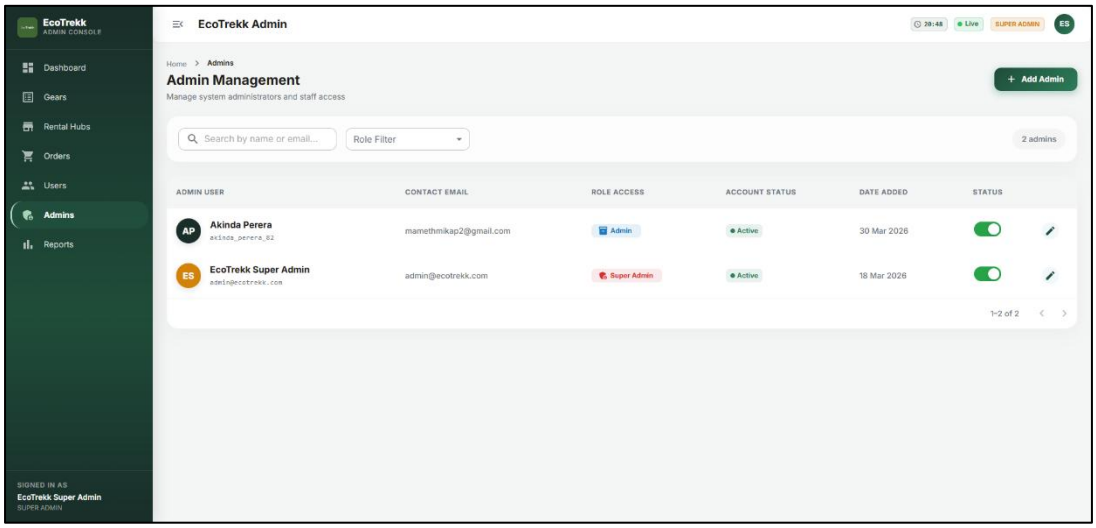


Figure 23: Admin Admin Management Page

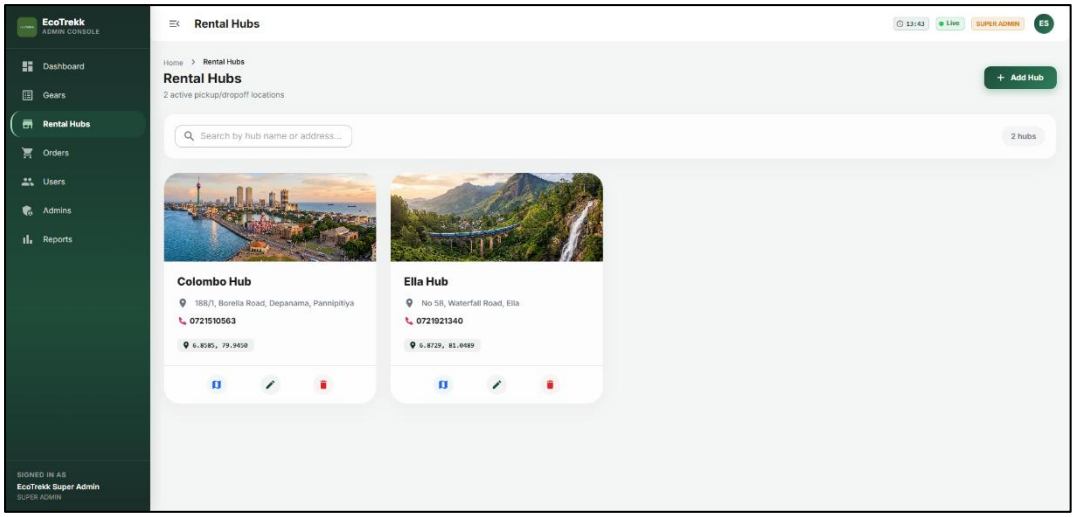


Figure 25: Admin Rental Hubs Management Page

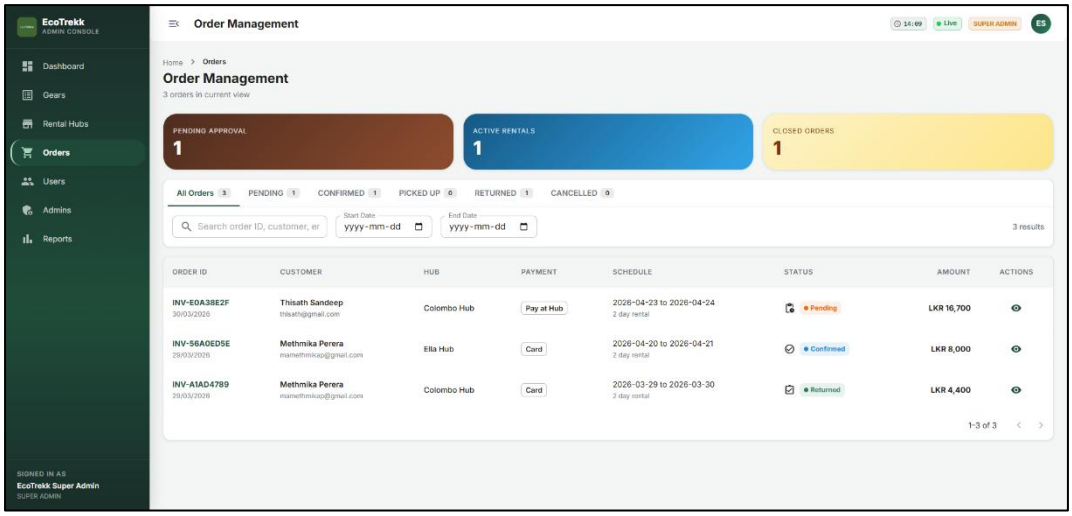


Figure 24: Admin Order Management Page

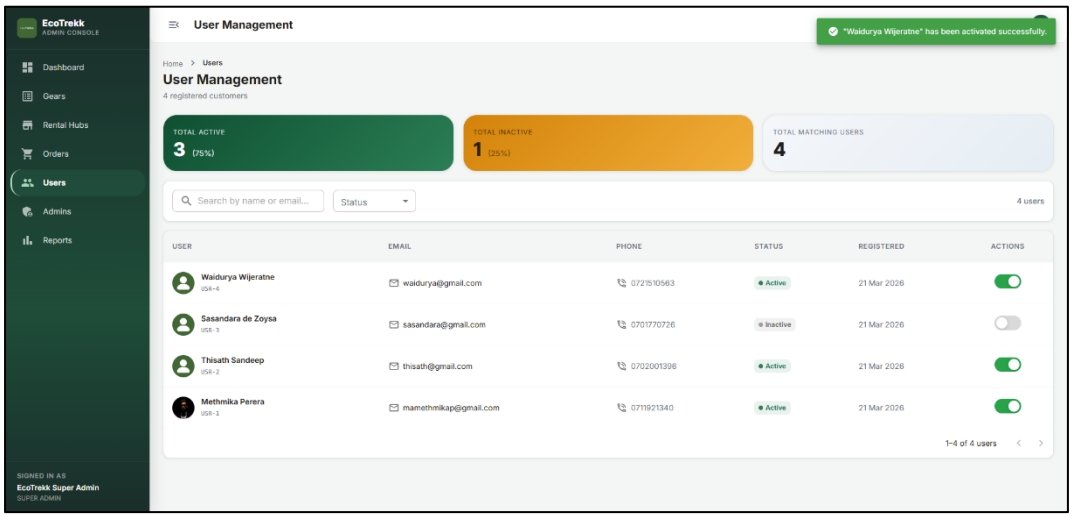


Figure 26: Admin Users Management Page

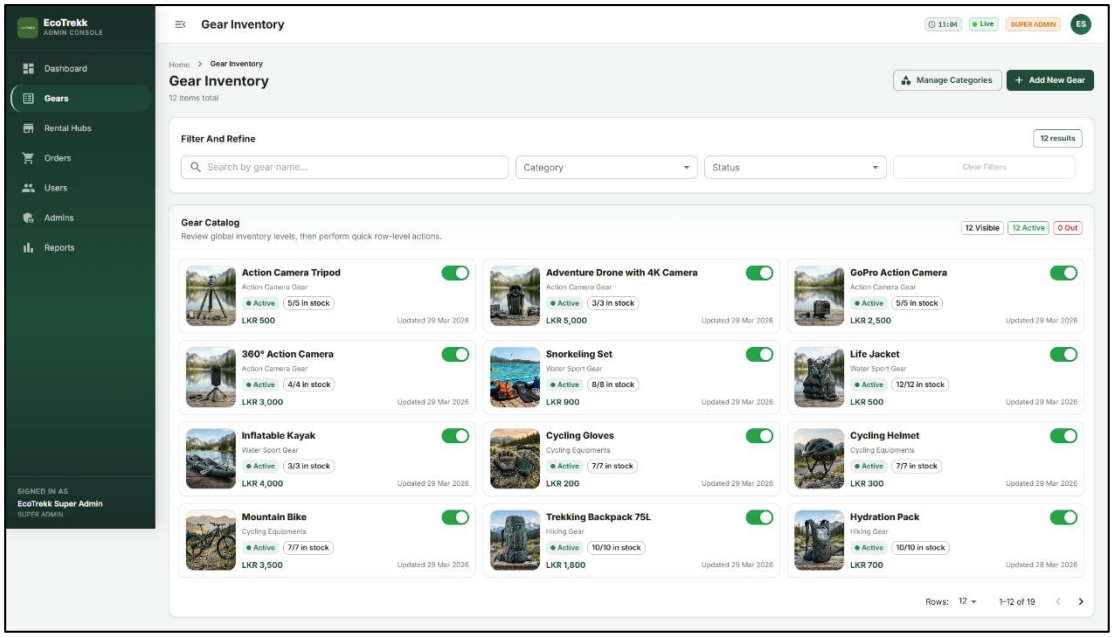


Figure 27: Admin Gear Management Page

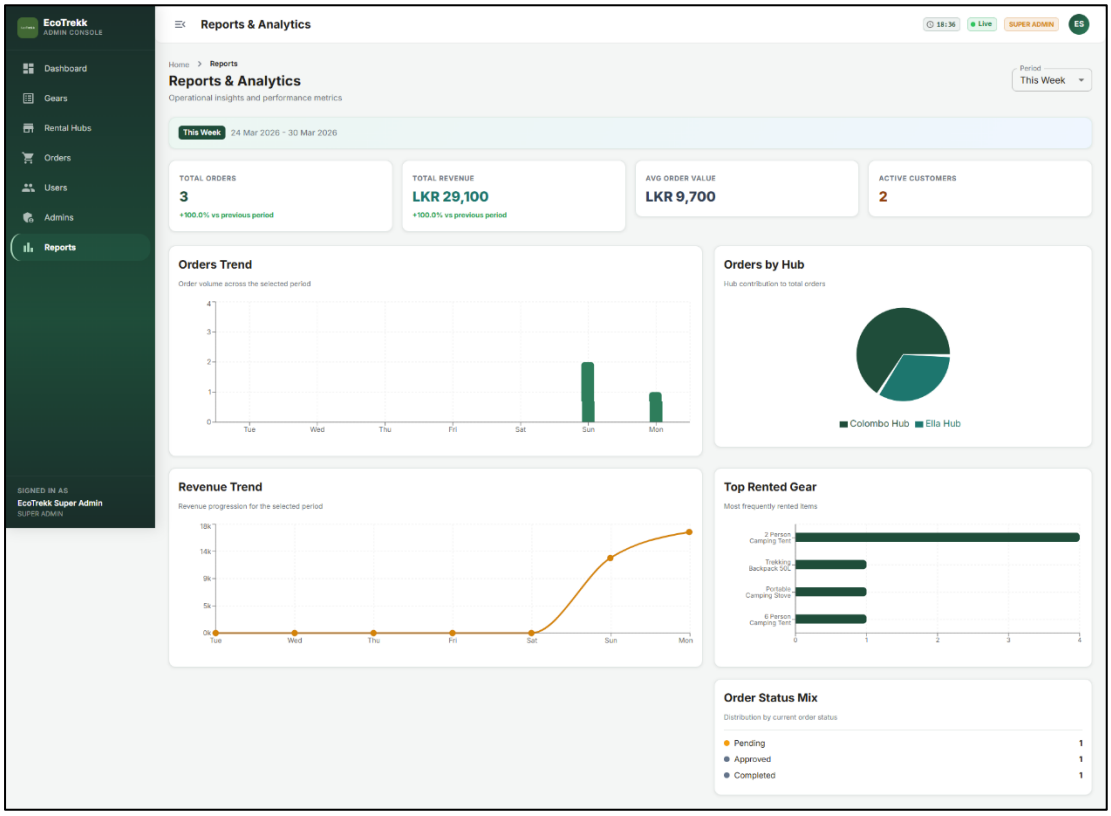


Figure 28: Admin Reports Page

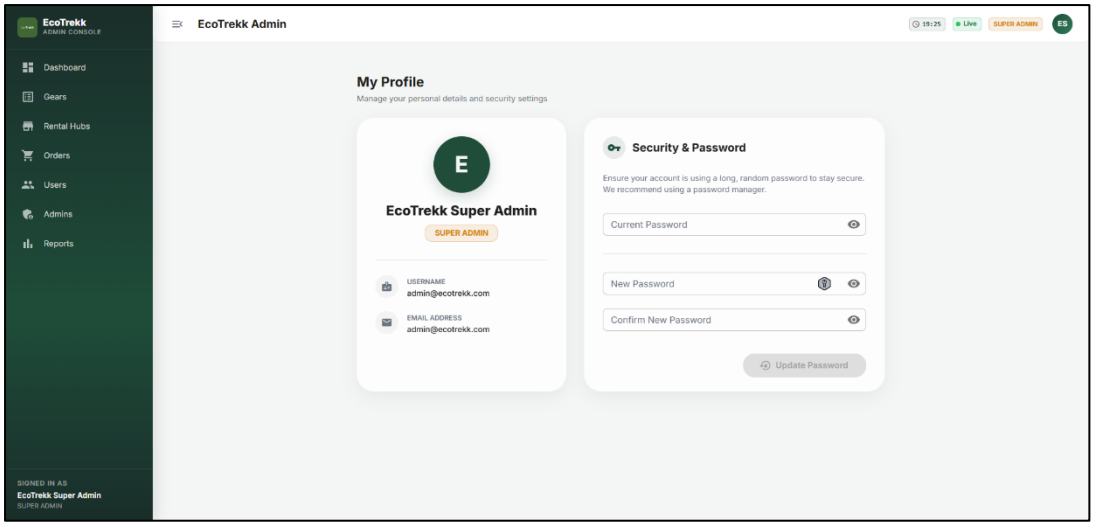


Figure 29: Admin Profile Page

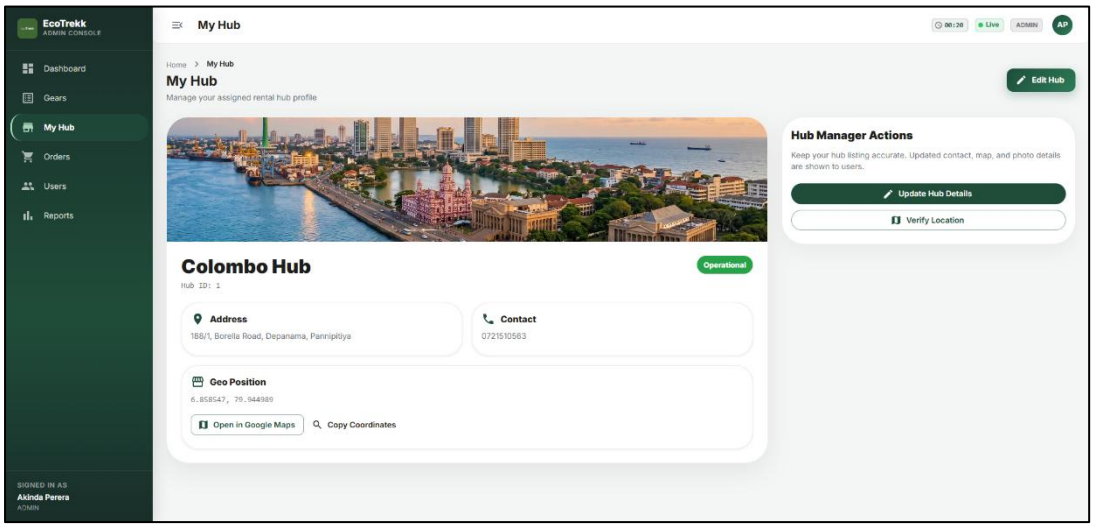


Figure 30: My Hub Page

5. Implementation:

5.1 Android Application Implementation

5.1.1 Live Data Retrieval with Retrofit

The Android application follows a repository pattern in which all data retrieval is performed by live HTTP calls to the Spring Boot backend via the Retrofit library. Each screen fetches its data from the corresponding backend endpoint on screen creation. Retrofit interfaces declare the API endpoints as annotated Java methods, and a Gson converter factory handles JSON deserialization into model objects. A custom OkHttp interceptor is responsible for attaching the current Firebase ID token as a Bearer token in the Authorization header of every outbound request. The interceptor retrieves the token synchronously using FirebaseAuth.getInstance().getCurrentUser().getIdToken(false).getResult(), with token refresh handled transparently by the Firebase SDK when the token is within its expiry window. A secondary OkHttp interceptor handles network error detection, catching IOException instances and posting an error event to a shared NetworkStateViewModel, which causes the UI to display a contextually appropriate connectivity error message and disables the checkout button to prevent submission failures. Responses are mapped through a generic ApiResponse wrapper class that encapsulates the HTTP status code, the deserialized body, and any error message. ViewModel classes observe LiveData objects populated by the repository and apply the appropriate UI state (loading, success, or error) to the fragment's view binding. This separation ensures that UI logic remains in the fragment layer while data logic remains entirely within the repository and ViewModel.

5.1.2 Biometric Authentication

Biometric authentication is integrated at the checkout stage using the AndroidX Biometric library, which provides a unified API over the platform's fingerprint and face recognition hardware. The implementation creates a BiometricPrompt instance bound to the CheckoutFragment's lifecycle, with a BiometricPrompt.AuthenticationCallback. Authentication success is used as a logical gate like if the callback's onAuthenticationSucceeded method is invoked, the checkout submission coroutine is resumed; if onAuthenticationError or onAuthenticationFailed is called, an error state is shown and the submission is not attempted.

5.1.3 Stripe Payment Integration

Stripe payment is integrated using the Stripe Android SDK. The checkout flow follows the recommended client-server pattern: the Android application first requests a PaymentIntent client secret from the backend (POST /api/orders), then uses the Stripe SDK's PaymentSheet component to present the payment UI and complete the transaction. This pattern ensures that the Stripe API secret key is never transmitted to the client device.

5.2 Spring Boot Backend Implementation

5.2.1 JWT Authentication Architecture

Authentication in the Spring Boot backend is handled by a custom JwtAuthenticationFilter that extends OncePerRequestFilter. For each incoming request, the filter extracts the Bearer token from the Authorization header and submits it to the Firebase Admin SDK's FirebaseAuth.verifyIdToken() method. Upon successful verification, the filter extracts the uid and email claims from the decoded token, loads the corresponding Customer record from the database, and constructs a UsernamePasswordAuthenticationToken that is placed in the Spring Security context. This design means that the backend performs no password management and no token issuance — all identity management is delegated to Firebase.

Role-based access control is enforced via Spring Security's method-level security annotations. Endpoints prefixed with /api/admin require the ROLE_ADMIN authority; endpoints prefixed with /api/manager require ROLE_MANAGER or ROLE_ADMIN. The Customer role is granted to all authenticated users by default.

5.2.2 Atomic Inventory Allocation

The prevention of concurrent double-booking is the most critical correctness requirement of the backend. The allocation logic is encapsulated in the InventoryAllocationService class, whose core method is annotated with @Transactional(isolation = Isolation.SERIALIZABLE). Within this transaction, the service acquires a pessimistic write lock on the InventoryItem record using Spring Data JPA's @Lock(LockModeType.PESSIMISTIC_WRITE) on the repository method. The allocation sequence is as follows:

1) The InventoryItem record is fetched with a pessimistic write lock, causing the database to block any concurrent transaction attempting to read or modify the same row;
2) The service verifies that the requested quantity does not exceed the current availableQuantity;
3) If the check passes, the availableQuantity is decremented by the requested amount and the record is flushed;
4) An OrderItem record is created linking the Order to the InventoryItem. If any step fails including if the concurrent lock check reveals insufficient stock, a custom InsufficientStockException is thrown, causing the entire transaction to roll back and ensuring that no partial allocation persists.

This strategy was selected over optimistic locking because the expected frequency of concurrent booking conflicts in the initial single-hub deployment is low, and pessimistic locking provides a simpler and more immediately comprehensible correctness guarantee. Optimistic locking would be more appropriate at scale, where lock contention would impose a measurable latency penalty.

5.2.3 WebSocket Integration

Real-time communication with the React dashboard is implemented using Spring's WebSocket support with the STOMP sub-protocol, configured via the WebSocketMessageBrokerConfigurer interface. The configuration enables a simple in-memory STOMP message broker with a destination prefix of /topic, and maps incoming STOMP messages to the /app prefix for controller routing. When a significant state change occurs (order created, payment confirmed, item returned, stock level changed) the InventoryEventPublisher service is invoked. This service uses Spring's SimpMessagingTemplate to broadcast a serialized JSON message to the relevant /topic/hub/{hubId}/inventory or /topic/orders destination. All connected React dashboard clients subscribed to that destination receive the message within the current WebSocket session without polling.

5.3 React Admin Dashboard Implementation

5.3.1 StompJS Integration

The React dashboard establishes a WebSocket connection to the Spring Boot backend using the @stomp/stompjs library. The connection is initialized in a useWebSocket custom hook that is invoked at the application root and whose client instance is made available to consuming components via a React Context provider. The hook configures the STOMP client with the WebSocket factory pointing to the back end's /ws endpoint, reconnect delay, and connection lifecycle callbacks. Upon successful STOMP connection (the onConnect callback), the hook subscribes to the relevant topic destinations based on the authenticated user's hub assignments. Messages received on subscribed destinations are dispatched to the Redux store via the appropriate slice action, causing the affected UI components to re-render with the new state without requiring a REST API call.

5.3.2 State Management with Redux Toolkit

Global state management in the React dashboard is handled by Redux Toolkit. Separate slices manage the hub, inventory, order, and authentication state domains. Async REST API operations are encapsulated in createAsyncThunk actions, which automatically generate pending, fulfilled, and rejected action types consumed by the corresponding slice reducers. WebSocket-delivered state updates are handled by synchronous slice actions issued directly from the useWebSocket hook, providing immediate state mutation without the expense of an asynchronous truncation.

5.3.3 Recharts Analytics Views

The analytics section of the dashboard employs Recharts components rendering against data fetched from dedicated backend analytics endpoints that return pre-aggregated time-series data. The charts are responsive, using the ResponsiveContainer wrapper to adapt to the available viewport width. Tooltip components are customised to display formatted currency values and precise date labels.

6. Results and Evaluation:

6.1 Inventory Lifecycle Testing

The system was tested by simulating the full lifecycle of equipment rentals. When an order is created, the backend automatically allocates inventory items and updates availability status. Upon completion of the rental and return of equipment, inventory is automatically restocked.

Table 1: Inventory Lifecycle Testing Outcomes

Test Case	Outcome
Single order, sufficient stock	PASS – Stock reduced correctly
Single order, insufficient stock	PASS – Error returned, stock unchanged
Order Rejected	PASS – Stock restored
Order Returned	PASS – Stock restored

6.2 Shake-to-Refresh Testing

The Android application's Shake-to-Refresh functionality was tested using the device accelerometer sensor. When a shake motion exceeding a predefined threshold was detected, the application triggered a refresh request to the backend.

6.3 Biometric Authentication Testing

Biometric authentication was tested on the physical test device using enrolled fingerprint credentials. The BiometricPrompt correctly presented the system fingerprint dialogue, accepted a valid fingerprint on the first attempt, and correctly gated the checkout submission. Authentication failure scenarios like incorrect fingerprint, user cancellation via the Cancel button were both handled correctly by the onAuthenticationError callback, displaying an appropriate error message and preventing order submission.

6.4 WebSocket Real-Time Update Testing

The real-time inventory update functionality was tested manually by opening the React dashboard in one browser window and the Android application on the test device simultaneously, then completing a checkout on the Android application. The dashboard's inventory table was observed to update the relevant row's available quantity without any page interaction within approximately 1-2 seconds of order creation, confirming the end-to-end WebSocket publication and delivery.

6.5 Future Testing Requirements

- Load Testing: Gatling or Apache JMeter load tests simulating 100-500 concurrent users performing simultaneous order creation against the Spring Boot API would establish the throughput and latency characteristics of the allocation engine and identify any connection pool exhaustion or deadlock conditions.
- **End-to-End UAT:** A structured UAT session with five to ten representative users navigating the complete rental journey — hub discovery, item selection, biometric checkout, and order tracking — would validate usability assumptions and uncover interface deficiencies not apparent during developer-led testing.
- **Security Penetration Testing:** A systematic review of the authentication token handling, JWT validation, and Stripe webhook signature verification should be conducted using tools such as OWASP ZAP to identify any exploitable vulnerabilities.
- **Concurrent API Testing:** Scenarios in which multiple customers attempt to book the same inventory items simultaneously from different devices should be tested end-to-end — including the full HTTP round-trip and Stripe payment flow — to confirm that the pessimistic locking strategy holds under real network conditions and that error responses are surfaced correctly in the Android UI.

7. Future Work:

Whilst EcoTrekk represents a functionally complete realisation of its stated specification, the project has surfaced a number of promising enhancement directions that were not feasible within the project's temporal and resource constraints. This section articulates those opportunities with appropriate technical specificity.

7.1 AI-Assisted Predictive Inventory Management

The current EcoTrekk system manages inventory reactively. Stock levels are updated in response to bookings and returns, but no mechanism exists to anticipate future demand. A natural and commercially valuable extension would be the integration of a predictive demand forecasting model, trained on the system's accumulated order history, seasonal patterns, local weather data, and regional event calendars. A time-series forecasting model such as Facebook Prophet or a lightweight LSTM neural network could be deployed as a microservice exposed to the Spring Boot backend. The back end’s administrative dashboard could then surface predictive restock alerts, recommending that hub managers increase stock levels for specific gear categories ahead of projected demand peaks.

7.2 Multi-Tenant Hub Architecture

The current architecture implicitly assumes a single rental business operating multiple hubs under unified administrative control. A commercially significant extension would be the introduction of a multi-tenant architecture, in which independent rental businesses (tenants) each manage their own set of hubs, inventory, customers, and financial data in complete isolation from one another, while sharing the same underlying infrastructure. This would require the introduction of a Tenant entity as the root of the data hierarchy, row-level security policies at the database level (or schema-per-tenant isolation), and tenant-aware routing logic in the Spring Boot security layer.

7.3 Cross-Platform Mobile Client

The current system is exclusively Android-native. Migration of the mobile client to a cross-platform framework such as Flutter or React Native, which would enable a single codebase to target both Android and iOS. Flutter's Dart language and its mature HTTP client (Dio) and state management (Riverpod) ecosystem provide a credible analogue to the current Retrofit and LiveData architecture.

8. Conclusions:

This report has presented EcoTrekk, a complete camping gear rental platform featuring a native Android mobile app, a Spring Boot RESTful API, and a React-based admin web dashboard. The system was designed to fill a clear gap in the outdoor equipment rental market: the lack of a unified digital platform that offers customers real-time inventory browsing, allows operators to manage stock live, and provides both parties with a secure and smooth transaction experience.

The specifications in Section 3 outlined requirements. The design in Section 4 addressed these requirements with a three-level distributed architecture, a normalized relational data model, a WebSocket-based real-time notification system, and a finite-state-machine model for the order lifecycle. The implementation, detailed in Section 5, turned these design choices into reality with specific technical decisions.

The evaluation in Section 6 showed that all main functional requirements were met and validated. The inventory allocation function successfully blocked concurrent over-booking in all test scenarios. Biometric authentication effectively controlled the checkout process on physical hardware. Real-time updates on the admin dashboard were delivered within about 1-2 seconds of changes in the backend state.

In summary, EcoTrekk offers a fully functional, well-structured, and commercially relevant solution to its identified problem. The project shows that it is possible to build a complex, multi-component rental platform within the timeframe of an academic project. It also provides a strong technical base for future development efforts.

9. Reflection on Learning:

Developing the EcoTrekk Camping Gear Rental System was a valuable learning experience:

- Full-Stack Development: Gained hands-on experience integrating Android, backend, and web technologies, learning to design and implement consistent API contracts, authentication flows, and real-time data synchronization.
- Agile and Iterative Approach: By iteratively refining requirements and design based on feedback and technical challenges, I improved my ability to adapt quickly and focus on user-centered solutions.
- Security Best Practices: Implementing secure authentication (Firebase, JWT) and data protection reinforced the importance of following industry standards, especially when handling sensitive user data.
- Scalability and Modularity: Early architectural decisions were revisited after encountering scalability bottlenecks. This led to a shift from a monolithic to a modular, service-oriented design, demonstrating double-loop learning by questioning and changing initial assumptions.
- Problem-Solving: I overcame challenges such as real-time synchronization, offline support, and deployment automation, which enhanced my troubleshooting skills and confidence in adopting new tools.
- User-Centric Mindset: Engaging with end-users during testing phases deepened my appreciation for usability, accessibility, and the need for clear feedback mechanisms.

Overall, this project fostered significant technical growth and encouraged a reflective, adaptive approach to software development, preparing me for future complex, cross-platform projects.

10. References:

Android Developers: https://developer.android.com/

Material3 Design Components: https://m3.material.io/components

Firebase Documentation: https://firebase.google.com/docs

Spring Boot Documentation: https://spring.io/projects/spring-boot

React Documentation: https://react.dev/

Material UI React Framework Documentation: https://mui.com/material-ui/getting-started/

Stripe Documentation: https://docs.stripe.com/

Google Maps SDK Documentation: https://developers.google.com/maps/get-started

Cloudinary Documentation: https://cloudinary.com/documentation

- END -